T A R E Q O

03 — Domaine

La couche `lib/domain/` est le **cœur** du système : règles métier pures, sans dépendance à Flutter, Dio, SQLite ou Riverpod. Elle contient :

Version 2.1.3+23 · 14 septembre 2026

TAREQO

Table des matières

Entités 3

- `Product` 3
- `StockLot` (W11 — lots FIFO) 3
- `Sale` + `SaleItem` 4
- `Debt` (« fiado »/crédit) 4
- `FinancialTransaction` 5
- `AppUser` (utilisateur authentifié) 5

Contrats de repository (interfaces) 6

Catalogue des cas d'usage 6

- Produits 6
- Stock (FIFO, W11) 7
- Ventes 7
- Finances 7
- Crédit (« fiado ») 7
- Authentification 8

Exceptions du domaine (catalogue) 8

Documents connexes 8

- Entités — objets immuables (Freezed) ;
- Contrats de repository — interfaces abstraites ;
- Cas d'usage — orchestration des règles métier ;
- Exceptions du domaine — erreurs métier levées par les cas d'usage.

Entités

Toutes les entités sont des classes @freezed (immuables, avec copyWith, ==/hashCode et sérialisation JSON via fromJson/toJson — voir build.yaml avec explicit_to_json: true).

> Phase 2 du multi-tenant : les entités métier (Product, Sale, FinancialTransaction, Debt, Customer, Purchase) portent `storeId` — le magasin auquel elles appartiennent (défaut 'st1'). Les appels getAll des repositories exigent storeId, donc chaque lecture/écriture est limitée au magasin actif. Voir Multi-tenant.

Product

Champ	Type	Notes
id	String	Clé primaire (générée avec AppIds.newId('p') — timestamp + entropie, sans collision)
storeId	String	Le magasin auquel il appartient (défaut 'st1' — Phase 2)
name	String	Obligatoire
description	String	Défaut ''
purchasePrice	int	Prix d'achat en centimes (W4)
salePrice	int	Prix de vente en centimes (W4)
stockQuantity	int	Défaut 0 — en milli-unités (W11 : 1000 = 1 unité/kg)
code	String?	Code-barres d'étiquette/PLU (W11, facultatif, 5 chiffres)
vendaPorFracao	bool	Défaut false — vendu au poids (fractions autorisées)
unitLabel	String	Défaut 'un' — unité de vente (kg/un)

Règle métier associée : salePrice >= purchasePrice — validée dans AddProductUseCase.

StockLot (W11 — lots FIFO)

Champ	Type	Notes
id	String	AppIds.newId('lot') (sans collision)
storeId	String	Le magasin du lot (défaut 'st1' — Phase 2)
productId	String	Produit auquel le lot appartient
entryDate	DateTime	Date d'entrée en stock

expiryDate	DateTime?	Péremption (facultatif — null = FIFO par date d'entrée)
initialQuantity	int	Quantité d'origine en milli-unités (W11)
currentQuantity	int	Quantité restante en milli-unités (W11)
costPriceCents	int	Coût d'acquisition par unité, en centimes

> Invariant : products.stock_quantity est toujours la somme des currentQuantity des lots actifs — garanti depuis la migration W11 (un lot initial lot-mig-… est créé pour chaque produit ayant du stock).

Sale + SaleItem

```
enum PaymentMethod {
 cash('Dinheiro'), debitCard('Cartão de débito'),
 creditCard('Cartão de crédito'), mPesa('M-Pesa'), eMola('E-Mola'),
 transfer('Transferência');
}
```

Sale	Type	Notes
id	String	AppIds.newId('s') (sans collision)
storeId	String	Le magasin de la vente (défaut 'st1' — Phase 2)
items	List<SaleItem>	Snapshot des articles vendus
totalAmount	int	Somme des articles au moment de la vente, en centimes (W4)
paymentMethod	PaymentMethod	Moyen de paiement
dateTime	DateTime	Moment de la vente

SaleItem	Type	Notes
productId	String	Traçabilité uniquement
productName	String	Snapshot du nom au moment de la vente
quantity	int	Quantité
unitPrice	int	Snapshot du prix au moment de la vente, en centimes (W4)

> Pourquoi des snapshots ? De futurs changements de prix/nom dans le catalogue ne peuvent pas corrompre l'historique : les rapports et audits utilisent la valeur facturée à l'époque, pas la valeur actuelle. Voir Flux métier.

Debt (« fiado »/crédit)

Champ	Type	Notes
id	String	AppIds.newId('d') (sans collision)
storeId	String	Le magasin de la dette (défaut 'st1' — Phase 2)

personName	String	Nom du client (ou fournisseur)
amount	int	@Assert('amount >= 0') — en centimes (W4)
dueDate	DateTime	Date d'échéance
isCustomerDebt	bool	true = à recevoir (client) ; false = à payer (fournisseur)
isPaid	bool	Défaut false — soldée ou non
paidAt	DateTime?	Date du règlement (alimente l'historique des règlements)

FinancialTransaction

```
enum TransactionType { income('Entrada'), expense('Saída') }
enum TransactionStatus { paid('Pago'), pending('Pendente') }
```

Champ	Type	Notes
id	String	AppIds.newId('t') (sans collision)
storeId	String	Le magasin de la transaction (défaut 'st1' — Phase 2)
description	String	Ex. : « Vente — M-Pesa », « Loyer du magasin »
amount	int	Toujours une magnitude positive (en centimes, W4) ; le sens vient de type
type	TransactionType	Entrée ou sortie
category	String	Ex. : « Vendas », « Contas », « Compras », « Fiado »
date	DateTime	Date de la transaction (date de réception dans le cas d'un règlement)
status	TransactionStatus	Défaut paid ; pending = entrée/« crédit » en attente
debtId	String?	Lien vers la Debt à l'origine de l'entrée en attente

> Entité séparée de `Sale` ? Oui, délibérément : toutes les entrées ne viennent pas d'une vente (apports de capital, intérêts) et toutes les sorties ne sont pas de la marchandise (loyer, électricité, salaires). Sale = événement opérationnel (articles/stock) ; FinancialTransaction = mouvement d'argent (sens/catégorie). Voir Flux métier.

AppUser (utilisateur authentifié)

Champ	Type	Notes
id	String	UID Supabase (ou placeholder avant confirmation de l'e-mail)
email	String	
fullName	String	Défaut '' — vient de user_metadata.full_name

> Abstrait l'utilisateur Supabase/gotrue User : le domaine et l'UI ne dépendent jamais du fournisseur d'authentification.

Contrats de repository (interfaces)

Ils vivent dans lib/domain/repositories/. Ils déclarent ce dont le système a besoin, jamais comment.

Contrat	Méthodes
IProductRepository	getAll(storeId), getById(id), add(product), updateStock(id, newQuantity)
ISaleRepository	registerSale(sale), getAll(storeId)
IDebtRepository	getAll(storeId), add(debt), markAsPaid(id)
IFinancialTransactionRepository	getByPeriod(storeId, start, end), getAll(storeId), add(transaction), markAsPaid(id)
ICustomerRepository	getAll(storeId), add(customer)
IPurchaseRepository	getAll(storeId), add(purchase)
IStockLotRepository	getByStore(storeId), getByProduct(storeId, productId), add(lot), updateQuantity(lotId, storeId, newQuantity)
IAuthRepository	signUp(...), signIn(...), signOut(), authStateChanges (Stream)
IUnitOfWork	run<T>(action) — exécute une action dans une transaction atomique

> Les méthodes de lecture (getAll/getByPeriod) prennent un `storeId` obligatoire : les modules métier sont limités au magasin actif (Phase 2).

Règles documentées dans les contrats :

- IDebtRepository.markAsPaid — ne doit pas être appelé directement depuis l'UI ; le règlement passe par SettleDebtUseCase (qui enregistre aussi l'entrée en caisse).
- IFinancialTransactionRepository.markAsPaid — la date devient la date de réception (la réalisation en caisse se produit quand l'argent entre).
- IProductRepository.updateStock — reçoit une valeur absolue ; la validation de non-négativité est la responsabilité du domaine (cas d'usage).

Catalogue des cas d'usage

Produits

Cas d'usage	Comportement
GetProductsUseCase	getAll(storeId) du repository.

AddProductUseCase	Valide salePrice >= purchasePrice ; lève InvalidPriceException (rien n'est persisté) avant add.
UpdateProductStockUseCase	Le seul point d'entrée pour muter le stock. ensureSufficientStock(productId, qty) (pré-validation sans mutation) et decrease(productId, qty) (revalide et persiste la valeur absolue stock − qty). Lève ProductNotFoundException / InsufficientStockException.

Stock (FIFO, W11)

Cas d'usage	Comportement
FifoStockUseCase	Diminution FIFO : pré-valide la somme des lots actifs avant toute mutation (InsufficientStockException), puis consomme les lots triés par expiry ASC → entryDate ASC → id (péremption la plus proche d'abord ; le surplus passe au lot suivant). increase crée un nouveau lot (entrée en stock) après avoir validé que le produit existe — pas de lots orphelins.
GetStockLotsUseCase	getByStore(storeId) / getByProduct(...) du repository — alimente les alertes de péremption.

Ventes

Cas d'usage	Comportement
ProcessSaleUseCase	Flux atomique complet (voir Flux métier) : valide la vente → pré-valide le stock → enregistre la vente → déduit le stock → entrée en caisse, le tout dans IUnitOfWork. Lève InvalidSaleException, InsufficientStockException, ProductNotFoundException.

Finances

Cas d'usage	Comportement
AddFinancialTransactionUseCase	Valide amount > 0 (InvalidAmountException) ; puis add.
GetCashFlowUseCase	Renvoie CashFlow(income, expense, balance) pour [start, end], en ne considérant que les transactions `paid` (caisse réalisée). Lève InvalidPeriodException si start > end.
MarkTransactionAsPaidUseCase	Marque une transaction en attente comme payée (bouton « Recevoir »).

Crédit (« fiado »)

Cas d'usage	Comportement
GetPendingDebtsUseCase	Filtre isCustomerDebt == true && !isPaid (à recevoir des clients).
GetSettledDebtsUseCase	Filtre isCustomerDebt == true && isPaid (historique des règlements).

AddDebtUseCase	Valide amount > 0 (InvalidDebtAmountException) ; enregistre la dette et crée l'entrée en attente liée (debtId) : entrée en attente pour les clients, sortie en attente pour les fournisseurs. Si l'entrée en attente échoue après la dette, lève PendingEntryRegistrationException (l'UI ne suggère pas de réessayer — cela dupliquerait le crédit).
SettleDebtUseCase	Solde la dette (marque isPaid + paidAt) et enregistre l'entrée en caisse (catégorie « Fiado »). Si une entrée en attente liée existe, marque la même transaction comme payée (pas de doublon). Idempotent — une dette déjà soldée ne génère pas de seconde entrée. Lève DebtNotFoundException. Ordre délibéré : règlement avant entrée (ne jamais facturer deux fois le client).

Authentification

Cas d'usage	Comportement
SignUpUseCase	Crée un compte (e-mail/mot de passe/nom).
SignInUseCase	Authentifie avec e-mail/mot de passe.
SignOutUseCase	Termine la session.

Exceptions du domaine (catalogue)

Exception	Levée par	Quand
InvalidPriceException	AddProductUseCase	Prix de vente < prix d'achat
InvalidSaleException	ProcessSaleUseCase	Vente sans articles ou total divergent
ProductNotFoundException	UpdateProductStockUseCase	Le produit n'existe pas
InsufficientStockException	UpdateProductStockUseCase	Stock insuffisant (avec available/requested)
InvalidAmountException	AddFinancialTransactionUseCase	Montant ≤ 0
InvalidPeriodException	GetCashFlowUseCase	Période inversée
InvalidDebtAmountException	AddDebtUseCase	Montant ≤ 0
PendingEntryRegistrationException	AddDebtUseCase	Crédit enregistré, entrée en attente en échec
DebtNotFoundException	SettleDebtUseCase	La dette n'existe pas
InsufficientStockException	FifoStockUseCase.decrease	Somme des lots actifs < quantité demandée (pré-validation, pas de diminution partielle)

Les écrans attrapent ces exceptions et affichent des messages conviviaux ; aucune exception d'infrastructure n'atteint l'UI.

Documents connexes

- Architecture — où le domaine s'insère.
- Flux métier — comment les cas d'usage se combinent en flux réels.
- Tests — cas d'usage testés avec des fakes de contrats.